

COMP 1324 Algorithmics

Tutorial 8

Hikmat Farhat

Question 1

- A delivery company has n trucks and a **single** team to load the trucks.
- Let f_i be the time needed by the team to load truck i and t_i be the time needed by truck i to deliver its content to the required destination **after** it is loaded.
- The trucks are loaded **sequentially** according to some schedule, but once loaded, a truck starts delivery independently of the others.
- Let T be the time when all the trucks have delivered their contents.
- Devise a greedy schedule for loading the trucks to minimize T . (Note that the team loads the trucks **one at a time**)

Question 1 solution

Let d_ℓ be the departure time of truck ℓ . All trucks will arrive at

$$A = \max_{1 \leq \ell \leq n} (d_\ell + t_\ell) \quad (1)$$

- Our objective is to minimise A . We assume by way of contradiction that greedy is not optimal.
- This means there is a pair of trucks i and $k = i + 1$ such that $t_k < t_i$ and k is scheduled before i .

We can write (1) as:

$$T = \max \left(\max_{\ell \neq i, k} (d_\ell + t_\ell) + \max(d_i + t_i, d_k + t_k) \right) \quad (2)$$

By swapping i and k , only the second term in (2) changes. We compute the value of the second term before and after the swap. Note that $t_i > t_k$.

Let d be the time it takes to load all the trucks before i and k .

option 1 i departs before k then

$$\begin{aligned} & \max(d_i + t_i, d_k + t_k) \\ &= \max(d + f_i + t_i, d + f_i + f_k + t_k) \\ &= d + f_i + \max(t_i, f_k + t_k) \end{aligned} \quad (3)$$

option 2 k departs before i then

$$\begin{aligned} & \max(d_i + t_i, d_k + t_k) \\ &= \max(d + f_i + f_k + t_i, d + f_k + t_k) \\ &= d + f_k + \max(f_i + t_i, t_k) \\ &= d + f_i + f_k + t_i \end{aligned} \quad (4)$$

There are two possibilities:

1. $t_i < f_k + t_k$. In this case, (3) give $d + f_i + f_k + t_k < (4)$
2. $t_i > f_k + t_k$. In this case, (3) give $d + f_i + t_i < (4)$

Therefore, in both cases, i departing first is the optimal choice.

Questions 2

- You are given the value of a certain stock at regular intervals in an array S .
- $S[0]$ is the initial value of the stock, $S[i]$ is the value at time i .
- At time i you are allowed to purchase maximum i shares of the stock for a total cost of $i \cdot S[i]$.
- Given an initial sum K , devise a greedy algorithm to purchase the largest number of stocks.

Question 2 solution

- The "value" to maximise is the number of shares.
- We would like to buy as many shares as possible so choose the largest number of shares per unit cost, i.e. $i/i \cdot S[i] = 1/S[i]$ first.
- Start by inserting the pairs $(i, S[i])$ into a priority queue Q with the key being $1/S[i]$.
- Then we can extract the maximum from Q and buy the maximum number of shares possible.
- We continue until we have no more money.
- The algorithm is as follows:

Question 2 solution

SD()

$rem \leftarrow K$

while $rem > 0$ **do**

$i \leftarrow \text{EXTRACT-MAX}(Q)$

if $i \cdot S[i] \leq rem$ **then**

$shares \leftarrow shares + i$

$rem \leftarrow rem - i \cdot S[i]$

else

$f \leftarrow \lfloor rem / S[i] \rfloor$

$shares \leftarrow shares + f$

$rem \leftarrow 0$

return $shares$

Question 3

- A certain monetary system has bills of values: 1,5,10,20.
- Given an amount x you need to find the smallest number of bills (repetition is allowed) that add up to the requested amount x .
 - 1 Give a greedy algorithm that finds the smallest number of bills that add up to x .
 - 2 Show that, by providing a counter example (specific value of x), the greedy approach no longer finds the optimal solution if an additional bill of value 25 is added to the system.
 - 3 Devise a dynamic programming strategy to find the optimal solution by writing down the recursive formula in terms of subproblems.

Question 3 solution

Let x be the initial amount and assume the values of the bills are in an array $B = [1, 5, 10, 20]$. The greedy algorithm is as follows:

```
rem  $\leftarrow x$ ;  
 $i \leftarrow \text{len}(B) - 1$ ;  
while  $\text{rem} > 0$  do  
    if  $B[i] \leq \text{rem}$  then  
         $\text{rem} \leftarrow \text{rem} - B[i]$ ;  
         $\text{bills} \leftarrow \text{bills} + 1$ ;  
    else  
         $i \leftarrow i - 1$ ;
```

Question 3 solution

- Consider $x = 40$ the greedy will use 25,10,5 instead of 20,20.
- The dynamic programming approach is as follows: Let $opt[i][r]$ is the minimal number of bills used when using the first i bills and the amount is r .
- Given r we have two choices: either we use bill i or we don't.
- If we use bill i then the number of bills used is $1 + opt[i][r - B[i]]$.
- If we don't use bill i then the number of bills used is $opt[i - 1][r]$. Therefore, the recursive formula is:

$$opt[i][r] = \min(opt[i - 1][r], 1 + opt[i][r - B[i]])$$

Question 4

- You need to climb a staircase with n steps.
- Each time you can move 1 or 2 steps.
- Let $c[n]$ be the number of ways you can climb the stairs. Obtain a recursive formula for $c[n]$.
- For example, if there are 3 steps you can climb the stairs in three different ways by taking: three 1-steps, one 1-step followed by one 2-steps, one 2-steps followed by one 1-step.

Question 4 solution

Solution

$$c[i] = c[i-1] + c[i-2]$$

Question 5

- Consider a grid of dimension $n \times n$.
- A robot needs to go from cell $(0, 0)$ to $(n - 1, n - 1)$.
- At each step the robot can move right or down (not left or up).
- Use dynamic programming to write a function that returns the total number of paths that the robot can take.

Question 5 solution

- In general, when the robot is at cell (i, j) it must have arrived either from $(i - 1, j)$ or from $(i, j - 1)$.
- Therefore, the unique paths it could have taken to arrive at (i, j) , $P[i][j] = P[i - 1][j] + P[i][j - 1]$.
- There are two "base" cases:
 - 1 At "row" 0, $P[0][j] = 1$ for all j since it could arrive only using j right moves
 - 2 At "column" 0, $P[i][0] = 1$ for all i since it could arrive only using i down moves

Question 5 solution

SD()

```
for  $i = 0$  to  $n$  do
|    $P[i][0] \leftarrow 1$ 
for  $j = 0$  to  $n$  do
|    $P[0][j] \leftarrow 1$ 
for  $i = 1$  to  $n$  do
|   for  $j = 1$  to  $n$  do
|        $P[i][j] \leftarrow P[i-1][j] + P[i][j-1]$ 
return  $opt$ 
```

Question 6

- You are supposed to manage the construction of billboards on a highway.
- The allowable positions, from some arbitrary starting point, are set by the government as $\{x_1, x_2, \dots, x_n\}$ with the restriction that billboards have to be at least 5 km apart.
- The profit from each billboard is given by the numbers $\{v_1, v_2, \dots, v_n\}$.
- For example, $X = \{8, 10, 12, 15, 16, 17, 20\}$ and $V = \{2, 7, 14, 9, 3, 13, 8\}$ then the optimal solution is to construct two billboard at positions 12 and 17 for a total profit of 27.
- Give a dynamic programming algorithm to find the optimized profit.

Question 6 solution

- Let $d(i) = \max_j \{j \mid x_i - x_j \geq 5\}$.
- Basically $d(i)$ is the largest index j such that $x_i - x_j \geq 5$.
- Given the last position x_n with profit v_n , there are two possibilities: either x_n belongs to the solution in which case, all x_j such that $j > d(n)$ cannot belong to the solution.
- The profit of this first case is $v_n + \text{opt}(d(n))$. If x_n does not belong to the solution then $\text{opt}(n) = \text{opt}(n-1)$. The recurrence relation we are looking for becomes

$$\text{opt}(i) = \max(\text{opt}(i-1), v_i + \text{opt}(d(i)))$$

Question 7

- ① Consider a **variation** of the knapsack problem (with replacement) where an item can be added to the knapsack **more than once**.
 - ① Given n items with values v_i and weights w_i , $i = 1 \dots n$ and a knapsack with capacity of C , obtain a recursive formula between the optimal solution and its subproblems.
 - ② Give the solution table for the instance $v = [2, 3, 4, 6]$, $w = [2, 3, 4, 5]$ with $C = 13$. Compare the solution with the 0-1 knapsack problem (with no replacement).

Question 7 solution

The only difference from the 0-1 knapsack problem is that we can add the same item more than once. The recursive formula is

$$\text{opt}[i][c] = \max(\text{opt}[i-1][c], v[i] + \text{opt}[i][c - w[i]])$$

Note that the second term has $\text{opt}[i][c - w[i]]$ instead of $\text{opt}[i-1][c - w[i]]$. A similar modification is made to the algorithm to reconstruct the solution.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	2	2	4	4	6	6	8	8	10	10	12	12
2	0	0	2	3	4	5	6	7	8	9	10	11	12	13
3	0	0	2	3	4	5	6	7	8	9	10	11	12	13
4	0	0	2	3	4	6	6	8	9	10	12	12	14	15

Question 8

- 2 A single processor needs to perform n tasks. Each task i has a duration t_i , a deadline d_i and only if the task is completed by the deadline it earns a profit p_i , otherwise the profit is zero. As an example, consider the following:

Name	Duration	Deadline	Profit
A	1	2	100
B	1	1	19
C	1	2	27
D	1	1	25
E	1	3	15

- Clearly in the above example, to earn the maximum profit you would schedule task A from 0 to 1 then task C from 1 to 2 and finally task E from 2 to 3 for a total earning of 142.
- The above case is simple, in that all durations are 1, and a greedy strategy where the highest profit is selected first works.

Question 8

Name	Duration	Deadline	Profit
A	3	8	100
B	2	6	59
C	1	9	40
D	2	5	70
E	4	10	101
F	5	10	140

- When the durations are different than 1, the greedy approach doesn't always work. For example, consider the example in the table above.
- A greedy strategy would give $\{F, E\}$ as solution with a profit of 241. The optimal solution, obtained using dynamic programming, is $\{C, D, A, E\}$ for a profit of 311.

Question 8 solution

- The basic idea is that each task reserves a time slot to execute, if possible.
- The constraint is that the task must be completed by its deadline and the time slot must be free.
- We start by sorting by deadline and obtain the maximum deadline.
- In the example above we get the order: D,B,A,C,E,F and a maximum deadline of 10. Any tasks that finishes after the maximum deadline will not earn any profit.

Then we ask: can the last task, in this case F, be in the solution? if yes then F "reserves" the $[5,10]$ interval and we consider the subproblem D,B,A,C,E with a new deadline of 5. If F is not in the solution then we consider the subproblem D,B,A,C,E with max deadline of 10.

Question 8 solution

In general, let $\delta[i], d[i], p[i]$ be the deadline, duration and profit of task i . for a given maximum deadline t and tasks $1 \dots n$ we have

```
 $tp \leftarrow \min(\delta[i], t) - d[i]$   
if  $tp < 0$  then  
    |  $opt[i][t] \leftarrow opt[i-1][t]$   
else  
    |  $opt[i][t] \leftarrow \max(opt[i-1][t], p[i] + opt[i-1][tp])$ 
```

Question 8 solution

Let T be the maximum deadline then the bottom up solution is:

SD()

```
for  $i = 1$  to  $n$  do
    for  $t = 0$  to  $T$  do
         $tp = \min(\delta[i], t) - d[i]$ 
        if  $tp < 0$  then
             $opt[i][t] \leftarrow opt[i - 1][t]$ 
        else
             $opt[i][t] \leftarrow \max(opt[i - 1][t], p[i] + opt[i - 1][tp])$ 
return  $opt$ 
```

As usual to reconstruct the solution we need walk backwards. $tasks$ is the sorted list of n tasks and T is the maximum deadline.

Question 8 solution

```
getTasks(opt)
```

```
i  $\leftarrow$  n
```

```
j  $\leftarrow$  T
```

```
while i > 0 and j > 0 do
```

```
    task  $\leftarrow$  tasks[i]
```

```
    if opt[i][j]  $\neq$  opt[i - 1][j] then
```

```
        | solution  $\leftarrow$  solution  $\cup$  task
```

```
        | j  $\leftarrow$  min( $\delta$ [i], j) - d[i]
```

```
    else
```

```
        | i  $\leftarrow$  i - 1
```

```
return solution
```

Python code can be found [here](#)